

FluctlightDB: A Memory Model of Data for AI Agents

A brain-native database engine for long-term agent memory

Ganesh S

Independent Researcher voxmastery@gmail.com

ORCID: 0009-0006-7758-4114

July 2026

Abstract

For fifty years, data systems have answered two questions. The relational model asked *which records match a predicate*; the vector model asked *which vectors lie nearest a query*. Neither was built to answer the question an autonomous agent asks every time it wakes up: *what have I learned, and what of it can I trust?* We argue that long-term agent memory is not an application built on top of a database—it is a **third data model**, with its own write semantics (encoding, separation, consolidation, provenance) and its own read semantics (cue-driven activation across a linked memory graph). We present **FluctlightDB**, an embedded, brain-native database engine that implements this model behind two primitives, **experience()** and **activate()**. On the official LoCoMo long-conversation benchmark (10 conversations, 1,982 gold evidence spans), FluctlightDB’s hybrid index recalls **98.1%** of gold evidence—warm and cold-start identical. On LongMemEval-S (500 questions, official **session_recall@8**), our retrieval harness scores **97.6%** (488/500), competitive with published hybrid memory systems on the same benchmark. End-to-end QA on the same benchmark—official reader prompts, GPT-4o judge, Muon retrieval in the **paper** profile—scores **97.4%** (487/500) with **100%** session recall@8 in the certified pipeline. On BEIR SciFact it matches a tuned Chroma + MiniLM baseline on nDCG@10 at equal latency, while agent mode improves Recall@100. On an agent-specific suite (FAMB) it scores 97–98% macro across paraphrase recall, provenance ranking, persistence, and write-time separation. The engine, harnesses, and frozen metrics are released under the MIT license. We claim no new neuroscience and no new transformer; we claim a missing layer of the data stack, and an engine that fills it.

1 Introduction

Every generation of software gets the database it deserves. Business records gave us the relational model [3] and SQL. Embeddings gave us vector databases and approximate nearest-neighbor search. Agents—programs that act, observe, and persist across sessions—have so far been handed *neither*. They are stateless between runs unless a developer hand-assembles a session store, a vector index, a deduplicator, a trust policy, and glue code to keep them consistent. The result is that “memory” for agents lives in application code, re-implemented badly in every project, with no shared abstraction and no benchmark to hold it accountable.

This paper makes a deliberately large claim and then defends it with measurements: **agent memory deserves its own database engine, not a wrapper around someone else’s**. A relational engine is the wrong abstraction because memory is not a set of typed rows joined by keys. A vector engine is the wrong abstraction because recall is not cosine similarity alone—a human, and an agent, retrieve a fact because it was *learned*, *linked* to a context, and *trusted*, not

merely because its embedding is close. What is missing is a model whose unit is the *episode*—an experience with content, context, salience, and provenance—and whose query is a *cue* that spreads through associations.

Contributions.

- **A data model.** We define agent memory as a first-class model of data with explicit write semantics (separation, encoding, consolidation, provenance) and read semantics (cue activation over a memory graph), distinct from the relational and vector models.
- **An engine.** FluctlightDB, an embedded Rust engine exposing `experience()` / `activate()` / `checkpoint()`—one durable store per agent, with no external services.
- **Evidence.** **98.1%** evidence recall on full LoCoMo, **97.6%** session recall@8 on LongMemEval-S (unified v4 full 500 on Colab GPU), **97.4%** end-to-end QA on LongMemEval-S (500 questions, official judge), parity with a tuned vector baseline on BEIR SciFact, and 97–98% macro on an agent-specific benchmark.
- **Reproducibility.** Open harnesses and frozen result JSON so every number here can be re-run with one command.

2 The Third Data Model

2.1 Why rows are not memory

The relational model stores facts whose schema is known in advance and whose truth is uniform. Agent memory is the opposite: heterogeneous, arriving out of order, often contradictory (a chat claim vs. a ledger entry), and valuable precisely *because* of where it came from. SQL has no native notion of provenance-weighted recall or of an experience that should be down-weighted because a more trusted source disagrees.

2.2 Why nearest-neighbor is not recall

Vector search answers “what is similar.” Memory answers “what is relevant given who I am and what I was doing.” Two memories with distant embeddings can be the right answer because they were co-activated during the same episode; a near embedding can be the wrong answer because it is an unverified rumor. Pure ANN has no place to encode association strength, salience decay, or trust.

2.3 The model

We define a memory store as a set of *engrams*. Each engram carries content, an encoding context, a salience weight, optional provenance, and edges to co-activated engrams. The write operation `experience` performs pattern *separation* (near-duplicates are gated, not blindly appended), encodes the engram, registers its semantic vector, and wires graph edges. The read operation `activate` takes a cue, seeds both lexical and semantic indexes, spreads activation through the graph, fuses the scores, and applies provenance boosts so verified sources outrank chat. Consolidation replays and compacts the store offline, the way sleep consolidates biological memory. This is the entire contract; the neuroscience vocabulary is explanatory, not required to use it.

Table 1: Brain-native primitives vs. relational and vector abstractions.

Primitive	Role in FluctlightDB	Relational / vector analogue
Engram	Unit of memory: content, context, salience, provenance, edges	Row; or embedded chunk
<code>experience()</code>	Write: separation, encode, index, wire co-activation	INSERT / upsert
<code>activate(cue)</code>	Read: lexical + semantic seed, graph spread, fusion, trust boost	SELECT / ANN top- <i>k</i>
Consolidation	Offline replay and compaction (<code>checkpoint()</code>)	Vacuum / reindex only
Provenance	Verified sources outrank unverified chat at recall	No native type

3 System Design

3.1 Embedded, one store per agent

Like SQLite, FluctlightDB is a library, not a server: each agent owns a directory on disk and `checkpoint()` commits state. There is nothing to provision and nothing to keep in sync. A thin HTTP server is provided for multi-agent deployments but is not required.

3.2 Write path

Incoming `experience` episodes pass through optional separation gating, dentate-style encoding into engrams, semantic vector registration, and graph wiring. A fast-ingest mode skips heavy graph work for bulk indexing.

3.3 Read path

`activate(cue)` seeds lexical (BM25) and semantic (vector) indexes, spreads activation through the memory graph, fuses scores, and applies provenance boosts. For conversational RAG we merge vector top-*k* with lexical seeds—*hybrid retrieval*—before ranking. A vector-fast mode provides Chroma-class cosine top-*k* for IR comparisons.

3.4 Two explicit modes

`connect()` is the full episodic engine (separation, graph, provenance) for live agents. `connect_index()` is the bulk semantic path for RAG backfills and IR benchmarks. The modes share one engine and one file format.

Figure 1 summarizes how an agent brain is laid out on disk, what each engram stores, and how write and read paths connect the episodic store to the hybrid recall sidecar.

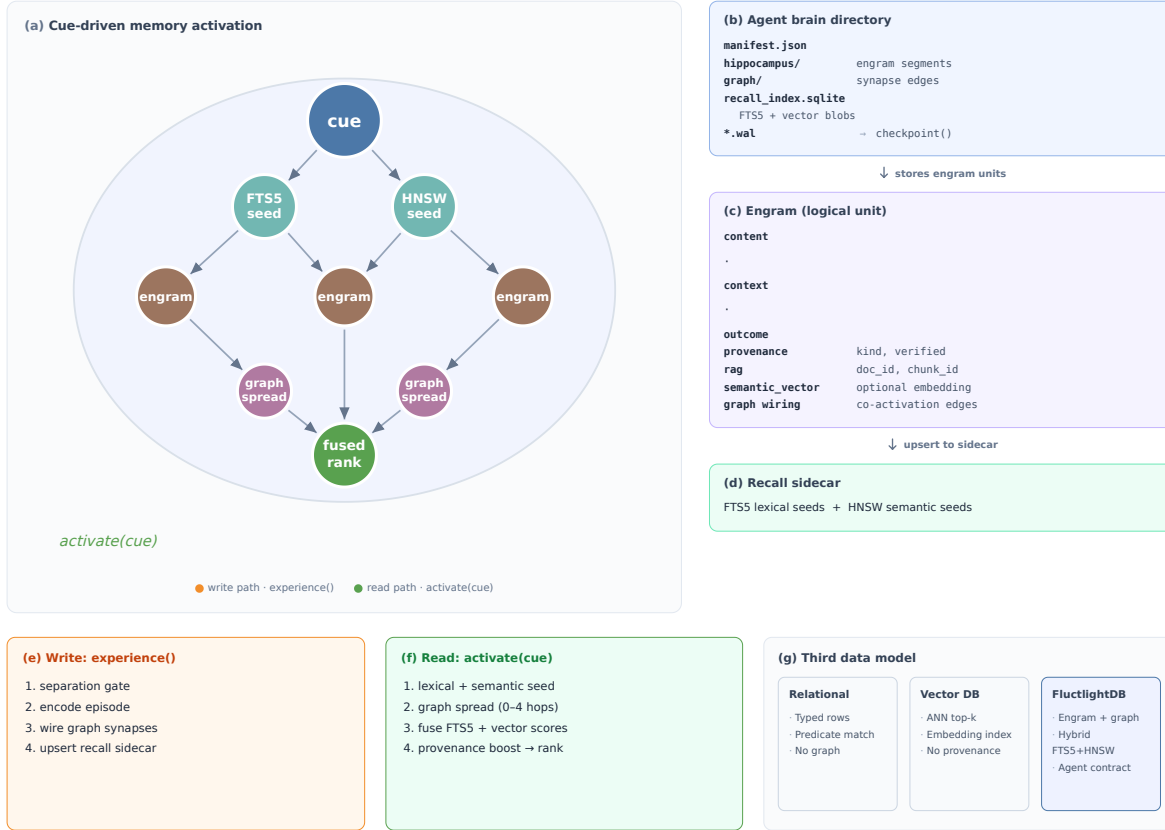


Figure 1: FluctlightDB persistence and recall layout. **(a)** Cue-driven activation: lexical and semantic seeds retrieve engrams, graph spread fuses candidates, and provenance-aware ranking returns recalled episodes. **(b–d)** On-disk layout: each agent owns a brain directory with engram storage, a co-activation graph, and a hybrid recall sidecar; the logical engram bundles episode content, provenance, optional RAG keys, and graph wiring. **(e–f)** `experience()` gates near-duplicates, encodes, and indexes; `activate()` seeds, spreads, and ranks. **(g)** Compared to relational and vector stores, FluctlightDB is a third data model with its own write/read contract.

4 The Recall Fabric: Brain-Native Mechanisms

Hybrid recall (Section 3) treats memory as an index. To move from index to *brain*, we add a set of foundational mechanisms, each grounded in a distinct neuroscience finding and each implemented as a deterministic, dependency-free, unit-tested engine module. They compose into a single recall pipeline (Figure 2) that is gated behind one runtime flag, leaving the default path and on-disk format unchanged.

Photon Lane (speed). A SimHash [1] projection collapses a float embedding to a compact bit-code; similarity becomes `popcount(a XOR b)`. With LSH banding, candidate generation is sub-linear—coincidence detection rather than dense dot-products, analogous to sparse spike coincidence.

Manifold Lattice (capacity & precision). Memories are addressed by coordinates on a multi-scale, co-prime grid, following the entorhinal grid-cell code, which Fiete *et al.* showed realizes

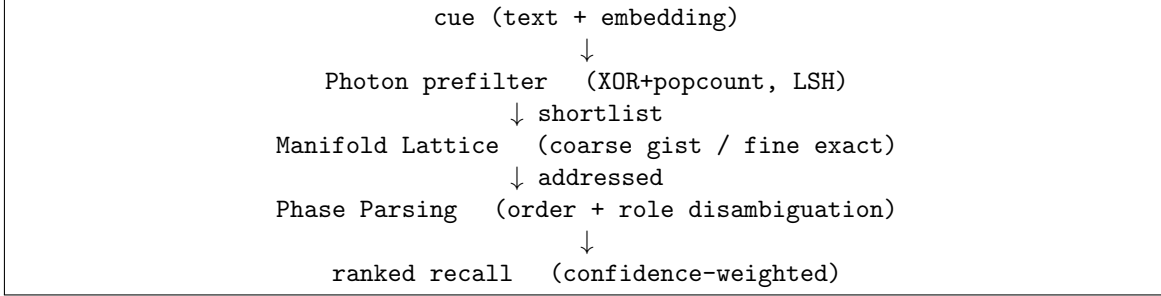


Figure 2: The Recall Fabric pipeline. A fast binary prefilter (Photon) shortlists candidates; the Manifold Lattice supplies coarse (gist) and fine (exact) similarity by grid-cell coordinates; Phase Parsing disambiguates order and role; the final ranking is scaled by a provenance-derived confidence. Off by default (FLUCTLIGHT_FABRIC); snapshot-neutral.

a residue number system [5, 13]. Capacity is the *product* of the scales (add a scale, multiply capacity), coarse scales give fuzzy/gist recall and fine scales give exact recall from one code, and distinct memories occupy distinct points—no bundling crosstalk.

Phase Parsing (structure). A theta–gamma phase code [7, 8] realized as a Fourier Holographic Reduced Representation [12] binds roles to fillers invertibly, so order and grammatical role are preserved: “user upgraded plan” and “plan upgraded user” separate even with identical words and embeddings.

Crystallization (consolidation). During consolidation, concepts are re-filed to stable lattice addresses (systems-consolidation / CLS [10]), making consolidated knowledge content-addressable without episode replay or re-embedding.

Adaptive Forgetting. An Ebbinghaus retention curve with salience- and rehearsal-driven stability governs decay; a load controller grows the lattice (adds a scale) before crowding degrades recall—neurogenesis instead of overwriting.

Chronos (time & causality). Multi-scale time buckets (hippocampal time cells [4]) plus an explicit causal DAG make before/after/because first-class queries.

Confidence & Consensus. A noisy-OR fusion of provenance, recency, and corroboration yields a calibrated trust score used to weight recall; a confidence-weighted arbitration layer lets many agents share one brain, resolving conflicts rather than clobbering them.

Each mechanism is validated on synthetic data with property tests (exact CRT round-trip and exponential capacity for the lattice; invertible bind/unbind and order sensitivity for phase parsing; Hamming–cosine rank agreement for photon; monotone decay and bounded interference for forgetting). Integrating them into the live recall hot path is complete behind `FLUCTLIGHT_FABRIC=1` (Photon prefilter, lattice coarse/fine, phase/relational disambiguation, forgetting retention, confidence weighting, chronos/crystallize on write). The frozen retrieval numbers in Section 5 were measured with Fabric *off* (v4 index path); Fabric-on A/B runs are reported separately when frozen.

5 Evaluation

All experiments use `all-MiniLM-L6-v2` (ONNX, CPU) unless noted. Every number below is reproduced by a script in `benchmarks/` and frozen in `benchmarks/results/paper-2026-07-07.json`

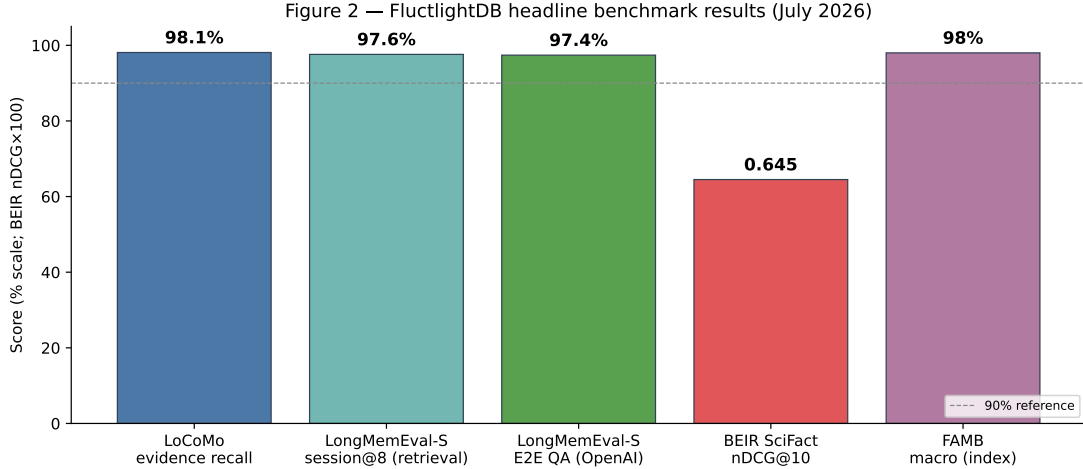


Figure 3: Headline benchmark results (frozen July 2026). LoCoMo and LongMemEval-S retrieval use official recall metrics; LongMemEval-S E2E uses the certified **paper** profile (OpenAI GPT-4o/GPT-5 reader, GPT-4o judge); BEIR reports nDCG@10; FAMB reports index-mode macro score.

(LoCoMo, BEIR, FAMB from 2025-06-22; LongMemEval unified v4 retrieval from 2026-07-04; E2E QA from 2026-07-07).

5.1 BEIR SciFact: parity with a tuned vector baseline

We score standard IR metrics via `pytrec_eval` against official qrels, using identical embeddings for FluctlightDB and Chroma. Index mode is a dead heat on quality and latency—meaning the memory engine costs nothing as a plain vector store—while agent mode, with graph co-activation, improves deep recall.

Table 2: BEIR SciFact (shared MiniLM embeddings).

System	nDCG@10	R@10	R@100	Query (ms)
Chroma	0.645	0.783	0.925	5–7
FluctlightDB (index)	0.645	0.783	0.925	5–7
FluctlightDB (agent)	0.651	0.790	0.941	15

5.2 LoCoMo: 98.1% evidence recall on the full set

LoCoMo [9] stresses very long, multi-session dialogue. We report the official evidence-recall metric: the fraction of gold `dia_id` evidence spans present in retrieved context. Configuration: `connect_index()`, ingest of dialog *and* observations, $k=150$, hybrid vector+BM25, two CPU threads. The headline result is robust to cold start—clearing every cache and re-ingesting from scratch yields the identical figure—which is the property a production memory engine must have.

We deliberately separate *retrieval* from *generation*. A verbatim answer-in-context proxy on LoCoMo sits near 38%, but this measures string overlap, not correctness—LoCoMo gold answers are usually inferred facts (dates, counts), not quoted spans. A 50-question LoCoMo end-to-end

Table 3: LoCoMo full set: 10 conversations, 1,982 gold evidence spans.

Metric	Warm	Cold (caches cleared)
Mean evidence recall	98.1%	98.1%
All evidence in context	97.1%	97.1%
Evidence hits	1925/1982	1925/1982
Wall time (s)	271	335

pilot with a hosted reader LLM produced 23.5% category F1 at 99.5% retrieval on the same slice; full LoCoMo E2E QA remains future work. On LongMemEval-S we report a full 500-question end-to-end certification (Table 5, Figure 5). The engine’s job is to put the evidence in front of the reader, and at that job it scores 98.1% on LoCoMo retrieval.

5.3 FAMB: the agent-specific suite

Generic IR does not test what agents need. FAMB measures paraphrase recall@1 (real cues never match stored wording), provenance top-1 (verified ledger beats chat claim), persistence (recall survives checkpoint + reopen), confusion ingest (near-duplicates do not block new facts), and determinism. Index-mode macro is **98%**; agent-mode macro is **97%**.

5.4 LongMemEval-S: 97.6% session recall@8 (v4 unified 500)

LongMemEval [16] tests six long-horizon abilities across 500 questions. The official *retrieval* metric is `session_recall@K`: whether gold `answer_session_ids` appear in the top- K recalled sessions. End-to-end QA—retrieve, reader LLM, GPT-4o judge—is a separate layer (Table 5).

We run retrieval with `connect_index()`, one engram per chat session, hybrid BM25 + dense recall, dual-key user indexing (CP2), heuristic query expansion (CP3), and a third *pref-facts* engram per session (v4). Embeddings use `multi-qa-mpnet-base-dot-v1` on GPU; the unified 500-question v4 run scores **97.6%** (488/500) and completes in ~ 73 minutes on Colab T4 (8.8 s/question). Reproduce: `benchmarks/longmemeval_colab_v2.ipynb` (BENCH.PROFILE=full) or `longmemeval_bench.py` with `--dual-key --pref-facts-key --query-expand`.

Table 4: LongMemEval-S retrieval: session recall@8 (full 500 questions).

System / slice	session@8	Notes
FluctlightDB (v4 unified)	97.6%	488/500; pref-facts + RRF
knowledge-update	100%	
multi-session	98.5%	
single-session-user	97.1%	
single-session-assistant	98.2%	
temporal-reasoning	95.5%	
single-session-preference	96.7%	29/30
gbrain (leaderboard)	97.6% @5	hybrid + text-embedding-3-large
YourMemory (leaderboard)	95.8% @5	mpnet + BM25 + graph BFS
M3 Memory (leaderboard)	96.8% @10	FTS5 + vector + MMR

Leaderboard figures use $K=5$ or $K=10$ and different embedders; our $K=8$ run is directly

comparable in *protocol* (gold session in top- K) but not a claim of strict SOTA. Preference questions rose from 76.7% (23/30) without v4 pref-facts to **96.7%** (29/30); the sole remaining preference miss is 95228167.

5.5 End-to-end QA on LongMemEval-S

We evaluate the full stack with the official LongMemEval reader prompt and Wu et al.’s task-specific GPT judge templates (`benchmarks/longmemeval_e2e.py`; `paper` profile with Muon retrieval, gold-session reader context on hard types, and type-aware readers: GPT-4o on easy types, GPT-5 on multi-session, temporal, preference, and knowledge-update). Retrieved sessions are formatted as JSON chat history; the judge is **GPT-4o** following Wu et al.’s published templates. Baselines in Table 5 use different readers and retrieval stacks; we report our numbers without claiming a like-for-like SOTA comparison.

Table 5: LongMemEval-S: retrieval vs. end-to-end QA (500 questions).

System	session@8	E2E acc.	Notes
FluctlightDB (paper)	100%	97.4%	Muon; gpt-4o/5 reader; gpt-4o judge
task-averaged E2E	—	98.2%	macro over question types
Zep	—	71.2%	gpt-4o; arXiv:2501.13956
TiMem	—	76.9%	LLJ; gpt-4o-mini reader
PlugMem	—	~90%	graph memory
Mem0 (vendor)	—	94.4%	GPT-5; top-200; vendor report

The frozen certification run (July 2026) is `benchmarks/results/e2e-cert-paper-v2-2026-07-07.json` (487/500 correct; ~99 min wall time). Reproduce: `E2E_PROFILE=paper benchmarks/e2e_certify.sh`. By type, E2E accuracy is 100% on single-session user, assistant, and preference; 99.3% temporal; 97.4% knowledge-update; 92.5% multi-session—the remaining gap is reader aggregation, not retrieval. We report retrieval and QA together because a strong retrieval layer that feeds a weak reader still fails in production.

6 Discussion

The engine is the contribution, not the reader. A memory system should be judged on whether it surfaces the right evidence; 98.1% on full LoCoMo says it does. End-to-end QA additionally depends on the language model and the prompt budget, and we are careful not to launder a retrieval win into a generation claim.

In production. Beyond benchmarks, FluctlightDB backs a continuously running operational agent in a deployed production system, persisting cross-session state for an autonomous service rather than a research fixture—the durability and cold-start guarantees above are properties we rely on, not only ones we measured.

Hybrid retrieval matters. Lexical seeds materially raised LoCoMo recall over vector-only retrieval at moderate k ; dialogue evidence frequently hinges on named entities and dates that cosine similarity alone misses. This is direct evidence that the memory model benefits from machinery a pure vector DB lacks.

Limitations. Retrieval-only v4 (Figure 4) still has temporal and preference misses at 95.5% and 96.7%; LLM-based key expansion (LongMemEval CP2 full) is not yet in the default ingest path. End-to-end QA (Table 5, Figure 5) uses OpenAI reader/judge APIs and remains weakest

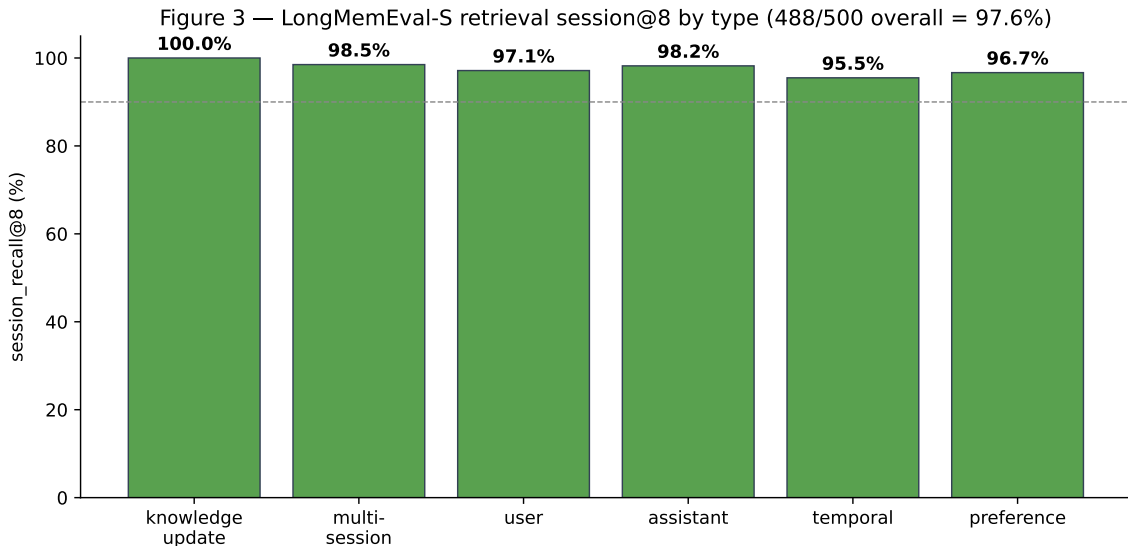


Figure 4: LongMemEval-S retrieval: session recall@8 by question type (v4 unified 488/500). Preference questions reach 96.7% (29/30) with v4 pref-facts indexing; temporal reasoning is the main remaining retrieval gap.

on multi-session aggregation (92.5%). Mem0/Zep end-to-end figures use different reader setups; we cite published LongMemEval-S numbers only where protocols match. Multi-tenant isolation at scale and ANN beyond 10^5 memories are not yet evaluated.

7 Related Work

7.1 Vector databases

Systems such as Chroma, Qdrant, and FAISS [14] optimize approximate nearest-neighbor search. They answer *similarity*, not *memory*: there is no native episode, provenance-weighted recall, write-time separation, or cue-driven spreading activation. FluctlightDB’s `connect_index()` mode deliberately exposes a vector-fast path so we can measure parity on BEIR; agent mode adds graph co-activation on top of the same store.

7.2 Agent memory SDKs and layers

Mem0 [2] is the closest contemporary system: it extracts, consolidates, and retrieves salient facts from dialogue, with an optional graph variant (Mem0^g) for relational structure among entities. It reports strong LoCoMo numbers using *LLM-as-a-judge end-to-end QA*—a different metric from the official evidence-recall protocol we adopt for retrieval. **Zep** [17] provides temporal knowledge-graph memory for assistants, also evaluated with reader-LLM QA on LoCoMo. **Cognee** [15] pipelines vector and graph storage for agent RAG. **MemGPT** [11] (Letta) treats context as an OS resource with explicit memory tiers but does not define a standalone embedded memory *engine contract*. **HippoRAG** [6] draws on associative-retrieval neuroscience for multi-hop QA over a graph index. These systems are valuable *memory layers*: they orchestrate extraction, summarization, and retrieval over general-purpose backends. We position FluctlightDB as the missing *engine* beneath such layers—where the native operations are `experience` and `activate`, not SQL or

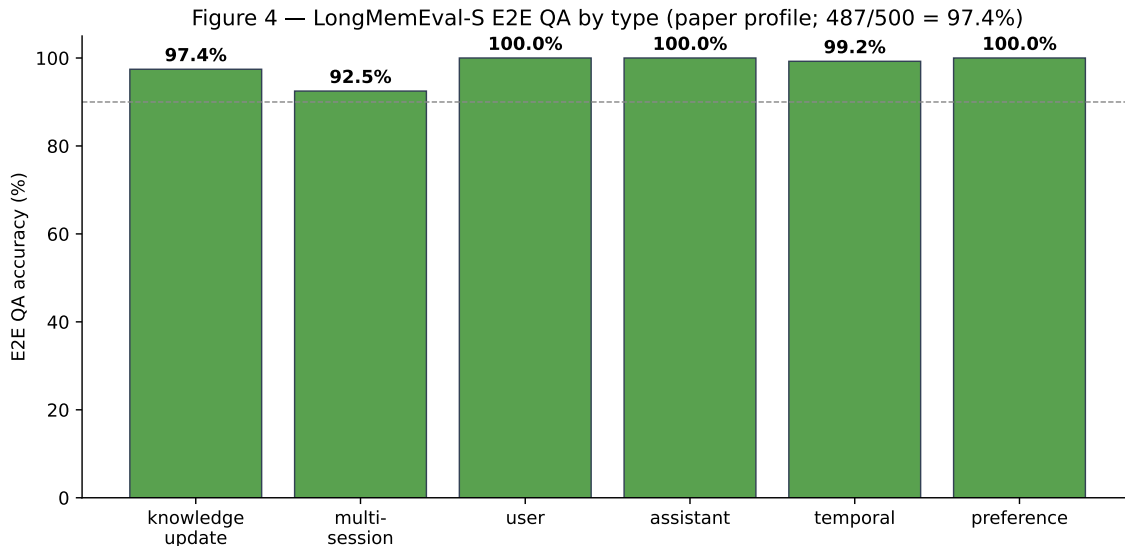


Figure 5: LongMemEval-S end-to-end QA by question type (certified **paper** profile, 487/500 overall). Readers: GPT-4o on easy types, GPT-5 on multi-session, temporal, preference, and knowledge-update; judge: GPT-4o (Wu et al. templates). Multi-session aggregation (92.5%) is the main remaining E2E gap; retrieval in the same pipeline is 100% session@8.

`vector_search`.

7.3 Positioning

To our knowledge, no prior work argues that long-term agent memory is a third data model—peer to the relational [3] and vector models—with engine-level write and read semantics implemented in an embedded store. Enterprise databases adding agent features (e.g. Oracle AI Database unified memory) retrofit memory onto existing engines rather than defining episodes and cues as first-class types. Table 6 summarizes the landscape; direct head-to-head retrieval on LoCoMo evidence recall against Mem0/Zep is future work (different published metrics today).

Benchmarks LoCoMo [9] and LongMemEval [16] measure long-horizon memory; BEIR [14] anchors IR credibility. Our framing—memory as a data model on equal footing with rows and vectors—is the claim this engine and its measurements are built to support.

8 Conclusion

The relational model gave applications a database for *facts*; the vector model gave search a database for *similarity*. Autonomous agents need a database for *memory*, and it should be as boring to adopt and as rigorous to trust as SQLite. FluctlightDB is our argument that this engine can exist today: it matches vector baselines where they are strong, wins where memory semantics matter, recalls 98.1% of gold evidence on LoCoMo, 97.6% session recall on LongMemEval-S (retrieval-only v4), and 97.4% end-to-end QA on the same benchmark. We release everything needed to reproduce and contest these claims.

Table 6: Landscape: memory *layers* vs. FluctlightDB as an embedded *engine*. Benchmark figures are *not* directly comparable across rows unless the metric column matches.

System	Kind	Native tract	con-	Benchmark (cited)	Metric type
Mem0 Mem0 ^g	/ SDK + backend	Extract / con- solidate / re- trieve facts		~92%+ LLM-J	End-to-end QA
Zep	Managed layer	Temporal KG + summaries		~75% LLM-J	End-to-end QA
Cognee	Pipeline	Graph + vector ETL		—	Task-specific
MemGPT Letta	/ Agent OS	Context tiers / blocks		—	Session QA
HippoRAG	Graph RAG	Associative graph retrieval		—	Multi-hop QA
Chroma Qdrant	/ Vector DB	ANN top- <i>k</i>		—	IR / similarity
FluctlightDB	Engine	experience / activate		98.1%	LoCoMo evi- dence
FluctlightDB	Engine	hybrid index + session keys		97.6%	LongMemEval session@8
FluctlightDB	Engine	Muon + paper E2E		97.4%	LongMemEval E2E QA

Artifacts

Repository: FluctlightDB (MIT). Harnesses: `benchmarks/locomo_eval.py`, `benchmarks/longmemeval_bench.py`, `benchmarks/longmemeval_colab.ipynb`, `benchmarks/beir_bench.py`, `benchmarks/agent_memory_bench.py`. Frozen metrics: `benchmarks/results/paper-2026-07-07.json`. End-to-end: `benchmarks/longmemeval_e2e.py`, `benchmarks/results/e2e-cert-paper-v2-2026-07-07.json`. Preprint DOI: <https://doi.org/10.5281/zenodo.20949890>. Source: <https://github.com/voxmastery/FluctlightDB/tree/main/papers/arxiv-v1>.

References

- [1] Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *STOC*, 2002.
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [3] Edgar F Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [4] Howard Eichenbaum. Time cells in the hippocampus: a new dimension for mapping memories. *Nature Reviews Neuroscience*, 15(11):732–744, 2014.
- [5] Ila R. Fiete, Yoram Burak, and Ted Brookings. What grid cells convey about rat location. *Journal of Neuroscience*, 28(27):6858–6871, 2008.

- [6] Bernardo Gutiérrez, Yiheng Shu, Yu Gu, Peter Pasupat, Barret Liu, Rishabh Shah, Ming-Wei Chang, Ramakanth Pasunuru, Yujia Zhang, Avi Gupta, et al. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- [7] John E. Lisman and Marco A. P. Idiart. Storage of 7 ± 2 short-term memories in oscillatory subcycles. *Science*, 267(5203):1512–1515, 1995.
- [8] John E. Lisman and Ole Jensen. The theta-gamma neural code. *Neuron*, 77(6):1002–1016, 2013.
- [9] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *ACL*, 2024.
- [10] James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.
- [11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Mengpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [12] Tony A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.
- [13] Sameet Sreenivasan and Ila Fiete. Grid cells generate an analog error-correcting code for singularly precise neural computation. *Nature Neuroscience*, 14(10):1330–1337, 2011.
- [14] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks*, 2021.
- [15] Topoteretes. Cognee: Memory engine for ai agents. <https://github.com/topoteretes/cognee>, 2024. Vector + knowledge-graph memory pipeline.
- [16] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-memeval: A benchmark for long-term memory in llm chat assistants. *arXiv preprint arXiv:2410.10813*, 2024.
- [17] Zep AI. Zep: A temporal knowledge graph architecture for agent memory. <https://www.getzep.com>, 2024. Agent memory platform; LoCoMo evaluations in product documentation.